

JAVA PROGRAMMING LABORATORY

INDUSTRY PROBLEM-SOLVING TASK

Java AWT, Applets, Event Handling and Layout Managers

Name of the Student	M. Madhu Sneha Latha	Register No.	192425092
Programmer / Section		Date	23/08/2026
Course Code	CSA0925	Course	Programming in Java
Faculty	Dr. Sivasankar	Duration	Lab Session

INDUSTRY TASK 1: Employee Performance & Payroll Dashboard

Problem Statement: A software company wants to develop a simple Employee Performance and Payroll Management System using Java AWT. The application should allow the HR team to enter employee details, process salary information, and generate a structured payroll report.

Functional Requirements

- Develop an AWT-based graphical user interface.
- Accept Employee ID, Employee Name, Department, Designation, Basic Salary, Working Days, and Performance Rating.
- Use suitable AWT components such as Label, TextField, Choice, Checkbox/CheckboxGroup, Button, TextArea, and List.
- Calculate Gross Salary, Performance Bonus, and Net Salary using suitable business rules defined by the student.
- Display the complete employee payroll report in a List or TextArea when Generate Report is clicked.
- Use GridBagLayout or BorderLayout for arranging the interface.
- Implement button events using ActionListener.
- Provide Submit/Generate Report and Reset functionality.
- Validate important inputs such as empty fields and numeric salary values.

Industry Context: The application simulates an HR payroll module used to capture employee information, perform basic payroll processing, and present a report for HR verification.

Expected Deliverables

- Java source code.
- Working GUI application with event handling.
- Sample input and output.
- Screenshot of the final application.
- Brief explanation of components, layout manager, and event handling used.

CODE:

```
import java.awt.Button;
import java.awt.Checkbox;
import java.awt.CheckboxGroup;
import java.awt.Choice;
```

JAVA PROGRAMMING LABORATORY

```
import java.awt.Color;
import java.awt.Frame;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.Label;
import java.awt.List;
import java.awt.TextArea;
import java.awt.TextField;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

public class EmployeePayrollDashboard extends Frame implements ActionListener {

    Label title;
    Label employeeIdLabel;
    Label employeeNameLabel;
    Label departmentLabel;
    Label designationLabel;
    Label basicSalaryLabel;
    Label workingDaysLabel;
    Label performanceLabel;

    TextField employeeIdField;
    TextField employeeNameField;
    TextField basicSalaryField;
    TextField workingDaysField;

    Choice departmentChoice;
    Choice designationChoice;

    CheckboxGroup performanceGroup;
    Checkbox excellent;
    Checkbox good;
    Checkbox average;
    Checkbox poor;

    Button generateButton;
    Button resetButton;

    TextArea reportArea;
```

JAVA PROGRAMMING LABORATORY

List employeeList;

```
public EmployeePayrollDashboard() {

    setTitle("Employee Performance & Payroll Dashboard");
    setSize(900, 650);
    setLayout(new GridBagLayout());
    setBackground(Color.LIGHT_GRAY);

    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    title = new Label("EMPLOYEE PERFORMANCE & PAYROLL DASHBOARD",
Label.CENTER);
    title.setBackground(Color.DARK_GRAY);
    title.setForeground(Color.WHITE);

    gbc.gridx = 0;
    gbc.gridy = 0;
    gbc.gridwidth = 4;
    add(title, gbc);

    employeeIdLabel = new Label("Employee ID:");
    employeeNameLabel = new Label("Employee Name:");
    departmentLabel = new Label("Department:");
    designationLabel = new Label("Designation:");
    basicSalaryLabel = new Label("Basic Salary:");
    workingDaysLabel = new Label("Working Days:");
    performanceLabel = new Label("Performance Rating:");

    employeeIdField = new TextField(20);
    employeeNameField = new TextField(20);
    basicSalaryField = new TextField(20);
    workingDaysField = new TextField(20);

    departmentChoice = new Choice();
    departmentChoice.add("IT");
    departmentChoice.add("HR");
    departmentChoice.add("Finance");
    departmentChoice.add("Marketing");
    departmentChoice.add("Operations");
```

JAVA PROGRAMMING LABORATORY

```
designationChoice = new Choice();
designationChoice.add("Software Engineer");
designationChoice.add("Manager");
designationChoice.add("Team Lead");
designationChoice.add("HR Executive");
designationChoice.add("Analyst");

performanceGroup = new CheckboxGroup();

excellent = new Checkbox("Excellent", performanceGroup, true);
good = new Checkbox("Good", performanceGroup, false);
average = new Checkbox("Average", performanceGroup, false);
poor = new Checkbox("Poor", performanceGroup, false);

gbc.gridwidth = 1;

gbc.gridx = 0;
gbc.gridy = 1;
add(employeeIdLabel, gbc);

gbc.gridx = 1;
add(employeeIdField, gbc);

gbc.gridx = 2;
add(employeeNameLabel, gbc);

gbc.gridx = 3;
add(employeeNameField, gbc);

gbc.gridx = 0;
gbc.gridy = 2;
add(departmentLabel, gbc);

gbc.gridx = 1;
add(departmentChoice, gbc);

gbc.gridx = 2;
add(designationLabel, gbc);

gbc.gridx = 3;
add(designationChoice, gbc);

gbc.gridx = 0;
```

JAVA PROGRAMMING LABORATORY

```
gbc.gridy = 3;
add(basicSalaryLabel, gbc);

gbc.gridx = 1;
add(basicSalaryField, gbc);

gbc.gridx = 2;
add(workingDaysLabel, gbc);

gbc.gridx = 3;
add(workingDaysField, gbc);

gbc.gridx = 0;
gbc.gridy = 4;
add(performanceLabel, gbc);

gbc.gridx = 1;
add(excellent, gbc);

gbc.gridx = 2;
add(good, gbc);

gbc.gridx = 3;
add(average, gbc);

gbc.gridx = 1;
gbc.gridy = 5;
add(poor, gbc);

generateButton = new Button("Generate Report");
resetButton = new Button("Reset");

generateButton.addActionListener(this);
resetButton.addActionListener(this);

gbc.gridx = 2;
gbc.gridy = 5;
add(generateButton, gbc);

gbc.gridx = 3;
add(resetButton, gbc);

Label reportLabel = new Label("Payroll Report:");
```

JAVA PROGRAMMING LABORATORY

```
gbc.gridx = 0;
gbc.gridy = 6;
gbc.gridwidth = 4;
add(reportLabel, gbc);

reportArea = new TextArea("", 10, 70, TextArea.SCROLLBARS_VERTICAL_ONLY);
reportArea.setEditable(false);
gbc.gridx = 0;
gbc.gridy = 7;
gbc.gridwidth = 3;
gbc.fill = GridBagConstraints.BOTH;
gbc.weightx = 1;
gbc.weighty = 1;
add(reportArea, gbc);

employeeList = new List(10);
gbc.gridx = 3;
gbc.gridy = 7;
gbc.gridwidth = 1;
gbc.weightx = 0.3;
add(employeeList, gbc);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        dispose();
    }
});

setVisible(true);
}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == generateButton) {
        generateReport();
    }

    if (e.getSource() == resetButton) {
        resetFields();
    }
}

public void generateReport() {
```

JAVA PROGRAMMING LABORATORY

```
String employeeId = employeeIdField.getText().trim();
String employeeName = employeeNameField.getText().trim();
String basicSalaryText = basicSalaryField.getText().trim();
String workingDaysText = workingDaysField.getText().trim();

if (employeeId.length() == 0 ||
    employeeName.length() == 0 ||
    basicSalaryText.length() == 0 ||
    workingDaysText.length() == 0) {

    reportArea.setText("Error: Please fill all required fields.");
    return;
}
double basicSalary;
int workingDays;
try {
    basicSalary = Double.parseDouble(basicSalaryText);
    workingDays = Integer.parseInt(workingDaysText);
} catch (NumberFormatException ex) {
    reportArea.setText(
        "Error: Basic Salary must be numeric and Working Days must be an integer."
    );
    return;
}
if (basicSalary <= 0) {
    reportArea.setText("Error: Basic Salary must be greater than 0.");
    return;
}
if (workingDays < 1 || workingDays > 31) {
    reportArea.setText("Error: Working Days must be between 1 and 31.");
    return;
}
String performance;

if (excellent.getState()) {
    performance = "Excellent";
} else if (good.getState()) {
    performance = "Good";
} else if (average.getState()) {
    performance = "Average";
} else {
    performance = "Poor";
}
```

JAVA PROGRAMMING LABORATORY

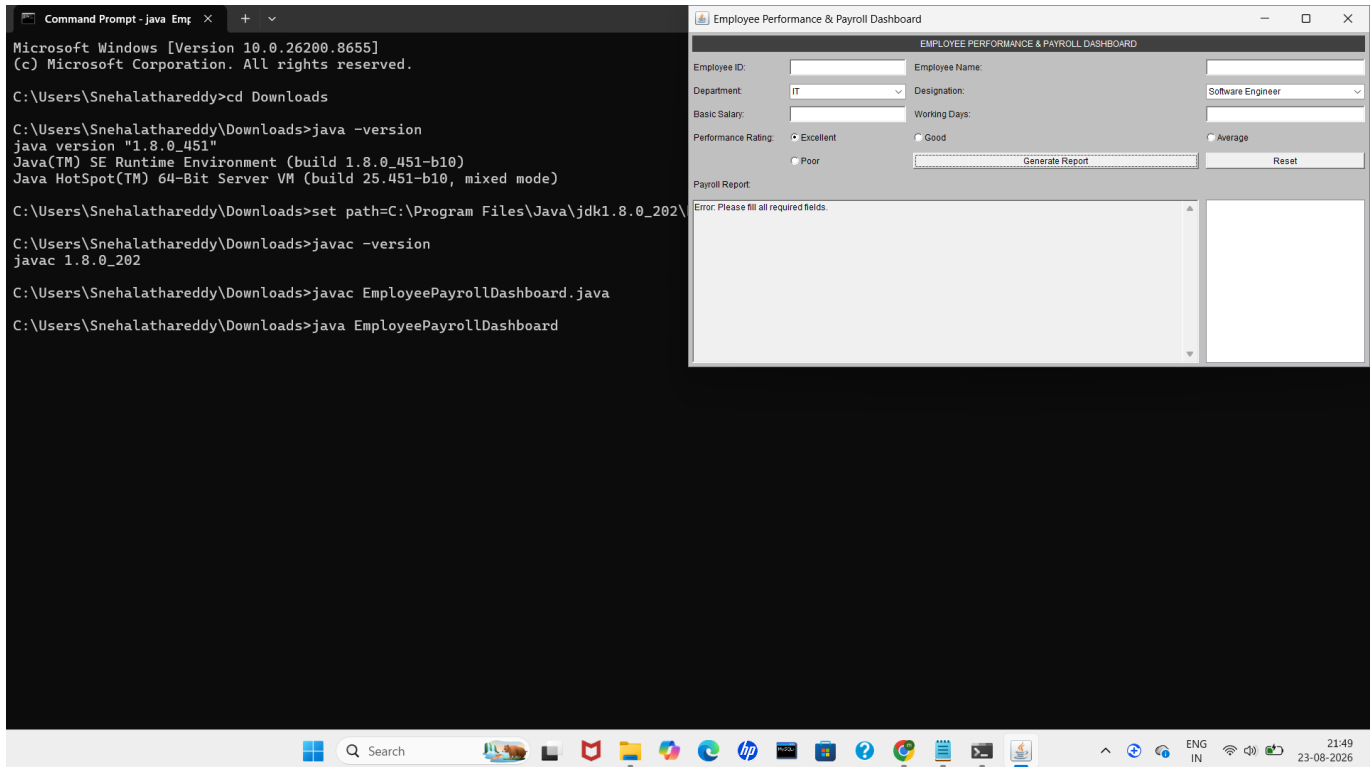
```
double grossSalary;
double bonusRate;
double performanceBonus;
double netSalary;
grossSalary = basicSalary * workingDays / 26.0;
if (performance.equals("Excellent")) {
    bonusRate = 0.20;
} else if (performance.equals("Good")) {
    bonusRate = 0.10;
} else if (performance.equals("Average")) {
    bonusRate = 0.05;
} else {
    bonusRate = 0.00;
}
performanceBonus = grossSalary * bonusRate;
netSalary = grossSalary + performanceBonus;
String report = "";
report += "=====\n";
report += "    EMPLOYEE PAYROLL REPORT\n";
report += "=====\n";
report += "Employee ID    : " + employeeId + "\n";
report += "Employee Name   : " + employeeName + "\n";
report += "Department     : " + departmentChoice.getSelectedItemAt() + "\n";
report += "Designation    : " + designationChoice.getSelectedItemAt() + "\n";
report += "Basic Salary    : Rs. " + String.format("%.2f", basicSalary) + "\n";
report += "Working Days    : " + workingDays + "\n";
report += "Performance Rating: " + performance + "\n";
report += "-----\n";
report += "Gross Salary    : Rs. " + String.format("%.2f", grossSalary) + "\n";
report += "Performance Bonus : Rs. " + String.format("%.2f", performanceBonus) + "\n";
report += "Net Salary      : Rs. " + String.format("%.2f", netSalary) + "\n";
report += "=====\n";
reportArea.setText(report);
employeeList.add(employeeId + " - " + employeeName);
}

public void resetFields() {
    employeeIdField.setText("");
    employeeNameField.setText("");
    basicSalaryField.setText("");
    workingDaysField.setText("");

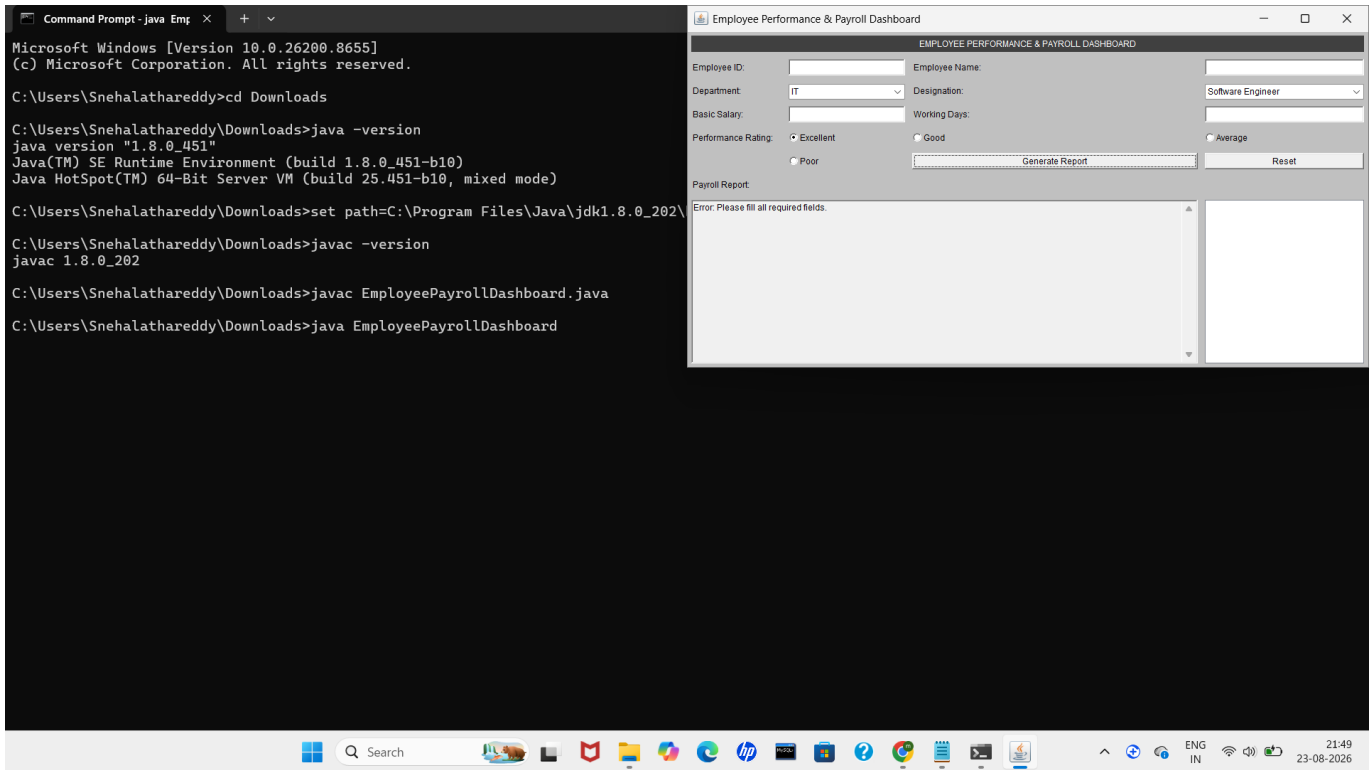
    departmentChoice.select(0);
```


JAVA PROGRAMMING LABORATORY

```
        designationChoice.select(0);
        performanceGroup.setSelectedCheckbox(excellent);
        reportArea.setText("");
        employeeList.removeAll();
    }
    public static void main(String[] args) {
        new EmployeePayrollDashboard();
    }
}
```



JAVA PROGRAMMING LABORATORY



Expected Deliverable 5 — Brief Explanation

- **Components Used**
- Label — displays field names such as Employee ID, Employee Name, Department, Salary, etc.
- TextField — accepts Employee ID, Employee Name, Basic Salary and Working Days.
- Choice — provides selectable options for Department and Designation.
- Checkbox — allows selection of the employee's Performance Rating.
- Button — provides Generate Report and Reset actions.
- TextArea — displays the generated payroll report.
- List — displays the employee ID and employee name after generating the report.
- Frame — acts as the main application window.

Layout Manager Used

- The application uses GridBagLayout.
- GridBagLayout arranges the components in rows and columns and allows different components to occupy different positions and sizes. It helps organize the employee input fields, buttons and payroll report neatly.

Event Handling Used

- The application implements ActionListener to handle button events.
- When the user clicks Generate Report, the actionPerformed() method calls generateReport(). The program validates the entered data, calculates the gross salary, performance bonus and net salary, and displays the payroll report.

JAVA PROGRAMMING LABORATORY

When the user clicks Reset, the `resetFields()` method clears the entered information and the generated report.

INDUSTRY TASK 2: Online Food Ordering and Billing System

Problem Statement: A restaurant wants to develop a simple Food Ordering and Billing Application for its counter staff. The application should allow customers to select food items, specify quantities, and generate a final bill.

Functional Requirements

- Develop the application using Java AWT.
- Create fields for Customer Name and Table Number.
- Provide Food Category, Food Item, and Quantity selection using appropriate AWT components.
- Provide Add Item, Generate Bill, Clear, Next, and Previous buttons.
- Display selected food items in a List or TextArea.
- Calculate item-wise cost, Subtotal, GST, and Final Bill Amount.
- Implement button click events using ActionListener.
- Use CardLayout to create separate screens for Customer/Order Details, Food Selection, and Bill/Order Summary.
- Validate quantity and ensure that an item is selected before adding it to the order.

Industry Context: The application represents a simplified restaurant POS (Point of Sale) system used for taking customer orders, processing prices, and generating a customer bill.

Expected Deliverables

- Java source code.
- Working multi-card AWT interface.
- Sample order and generated bill.
- Screenshot of each card/screen.
- Brief explanation of CardLayout, AWT components, and event handling.

CODE:

JAVA PROGRAMMING LABORATORY

Command Prompt - java Foo X + -
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Snehalathareddy>cd Downloads

C:\Users\Snehalathareddy\Downloads>java -version
java version "1.8.0_451"
Java(TM) SE Runtime Environment (build 1.8.0_451-b10)
Java HotSpot(TM) 64-Bit Server VM (build 25.451-b10, mixed mode)

C:\Users\Snehalathareddy\Downloads>set path=C:\Program Files\Java\jdk1.8.0_202\bin;%path%;

C:\Users\Snehalathareddy\Downloads>javac -version
javac 1.8.0_202

C:\Users\Snehalathareddy\Downloads>javac FoodOrderingSystem.java

C:\Users\Snehalathareddy\Downloads>javac FoodOrderingSystem
error: Class names, 'FoodOrderingSystem', are only accepted if annotation processor is enabled
1 error

C:\Users\Snehalathareddy\Downloads>java FoodOrderingSystem

C:\Users\Snehalathareddy\Downloads>java FoodOrderingSystem

Online Food Ordering and Billing System
CUSTOMER / ORDER DETAILS

Customer Name:

Table Number:

ClearNext

27°C
Mostly cloudy

Search

hp

22:15
23-08-2026

Command Prompt - java Foo X + -
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Snehalathareddy>cd Downloads

C:\Users\Snehalathareddy\Downloads>java -version
java version "1.8.0_451"
Java(TM) SE Runtime Environment (build 1.8.0_451-b10)
Java HotSpot(TM) 64-Bit Server VM (build 25.451-b10, mixed mode)

C:\Users\Snehalathareddy\Downloads>set path=C:\Program Files\Java\jdk1.8.0_202\bin;%path%;

C:\Users\Snehalathareddy\Downloads>javac -version
javac 1.8.0_202

C:\Users\Snehalathareddy\Downloads>javac FoodOrderingSystem.java

C:\Users\Snehalathareddy\Downloads>javac FoodOrderingSystem
error: Class names, 'FoodOrderingSystem', are only accepted if annotation processor is enabled
1 error

C:\Users\Snehalathareddy\Downloads>java FoodOrderingSystem

C:\Users\Snehalathareddy\Downloads>java FoodOrderingSystem

Online Food Ordering and Billing System
FOOD SELECTION

Food Category:

Food Item:

Quantity:

Add Item

Selected Items:

Pizza x 2 = Rs.500.00

PreviousNext

Clear

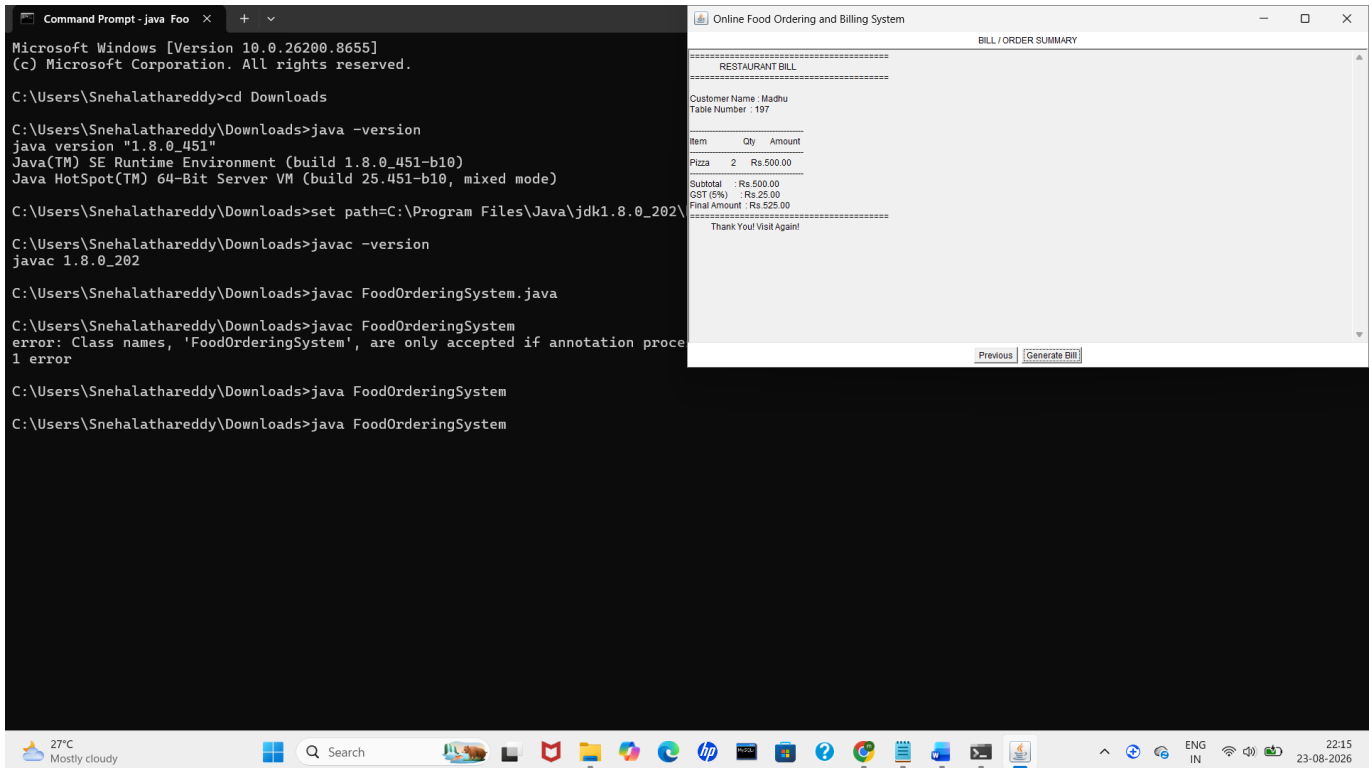
27°C
Mostly cloudy

Search

hp

22:15
23-08-2026

JAVA PROGRAMMING LABORATORY



Brief Explanation of CardLayout

CardLayout is a Java AWT layout manager that allows multiple panels or screens to occupy the same area. Only one card is visible at a time. In this application, three cards are created: Customer Details, Food Selection, and Bill/Order Summary. The CardLayout methods are used to move between these screens using the Next and Previous buttons.

In the program:

```
cardLayout = new CardLayout();
mainPanel = new Panel(cardLayout);
The cards are added as:
mainPanel.add(container, "customer");
mainPanel.add(container, "food");
mainPanel.add(panel, "bill");
```

Navigation is performed using:

```
cardLayout.show(mainPanel, "food");
and:
cardLayout.show(mainPanel, "bill");
```

Brief Explanation of AWT Components

Write:

AWT (Abstract Window Toolkit) provides GUI components used to create the user interface of the application.

The major AWT components used are:

AWT Component

Purpose

Frame: Creates the main application window

Panel: Groups and organizes components

Label: Displays text and field descriptions

TextField: Accepts customer name and table number

Choice: Provides dropdown selections

Button: Performs actions such as Next, Previous, Add Item and Generate Bill

TextArea: Displays selected items and the final bill

Dialog: Displays validation/error messages

CardLayout: Controls multiple screens/cards

8. Brief Explanation of Event Handling

Write:

Event handling is used to perform actions when the user interacts with GUI components such as buttons and dropdowns.

The program implements:

implements ActionListener

The method:

public void actionPerformed(ActionEvent e)

handles button events.

For example, when the user clicks Add Item, the selected food item and quantity are added to the order.

When the user clicks Next, the program moves to the next card.

When the user clicks Previous, it returns to the previous card.

The category dropdown uses ItemListener to update the available food items according to the selected category.

For example:

Main Course
↓
Chicken Biryani
Veg Biryani
Fried Rice

and:

Drinks
↓
Coke
Fresh Juice
Coffee